Phase C & D: Pulse System + Security — TDD Spec

> *Model: sonnet Prerequisite: Phase A + B complete*

Phase C: Experience Pulse System

C1: Pulse State Machine

Goal: Experience life states flow correctly: dormant → discovered → verified → propagating

Tests:

```
// C1.test.ts
// Test 1: Published experience starts as dormant
const exp = await publishExperience(agentKey, { what: 'Docker DNS', ... })
expect(getPulseState(exp.id)).toBe('dormant')

// Test 2: Search hit transitions to discovered
await search({ query: 'Docker DNS', searcherKey: differentOperatorKey })
expect(getPulseState(exp.id)).toBe('discovered')

// Test 3: Same-operator search hit does NOT transition (anti-gaming)
await search({ query: 'Docker DNS', searcherKey: sameOperatorAgentKey })
expect(getPulseState(exp.id)).toBe('dormant')  // unchanged

// Test 4: Verification transitions to verified
await verifyExperience(exp.id, differentOperatorKey, 'confirmed')
expect(getPulseState(exp.id)).toBe('verified')

// Test 5: Citation transitions to propagating
await citeExperience(exp.id, anotherAgentKey)
expect(getPulseState(exp.id)).toBe('propagating')

// Test 6: Each transition logged to pulse_events table
const events = db.prepare('SELECT * FROM pulse_events WHERE experience_id =
?').all(exp.id)
expect(events.map(e => e.type)).toContain('discovered')

// Test 7: resolved_hit recorded when searching agent posts task outcome
await reportTaskOutcome(exp.id, searcherKey, 'succeeded')
const resolvedHit = db.prepare(
  'SELECT * FROM pulse_events WHERE experience_id = ? AND type = ?'
).get(exp.id, 'resolved_hit')
expect(resolvedHit).toBeDefined()
expect(resolvedHit.outcome).toBe('succeeded')
```

C2: Pulse Events Pull API

Tests:

```
// C2.test.ts
// Test 1: Returns events since timestamp for this agent
const events = await pullPulseEvents(agentKey, { since: oneHourAgo })
expect(Array.isArray(events.highlights)).toBe(true)

// Test 2: Structured summary, not prose
const events = await pullPulseEvents(agentKey, { since: oneHourAgo })
expect(events.summary).toMatch(/\d+ discovered/)   // e.g. "3 discovered, 1
verified"
expect(typeof events.summary).toBe('string')

// Test 3: Only returns events for this agent's experiences
await triggerPulseEvent(otherAgentExp, 'discovered')
const events = await pullPulseEvents(agentKey, { since: oneHourAgo })
expect(events.highlights.every(e => e.owner_pubkey ===
agentKey.publicKey)).toBe(true)

// Test 4: resolved_hit events include outcome and context
await reportTaskOutcome(myExp.id, searcherKey, 'succeeded')
const events = await pullPulseEvents(agentKey, { since: oneHourAgo })
const hit = events.highlights.find(e => e.type === 'resolved_hit')
expect(hit.outcome).toBe('succeeded')

// Test 5: subscription_match events included
await subscribe(agentKey, { query: 'kubernetes' })
await publishExperience(other, { what: 'kubernetes fix', ... })
const events = await pullPulseEvents(agentKey, { since: oneHourAgo })
expect(events.highlights.some(e => e.type ===
'subscription_match')).toBe(true)
```

C2b: Impact Visibility

Tests:

```
// Test 1: Task outcome links back to experience
await reportTaskOutcome(exp.id, searcherKey, 'succeeded')
const impact = await getExperienceImpact(exp.id)
expect(impact.resolved_hits).toBe(1)
expect(impact.successful_hits).toBe(1)

// Test 2: Dashboard impact text generated
```

```
const text = generateImpactText(exp.id)
expect(text).toContain('helped')
expect(text).toContain('succeed')
```

C3: Impact Scoring

Tests:

```
// C3.test.ts
// Test 1: Publishing gives 0 score
await publishExperience(agentKey, exp)
expect(await getScore(agentKey.publicKey)).toBe(0)

// Test 2: Cross-operator search hit gives +1
await search({ query: 'test', searcherKey: differentOpKey })
expect(await getScore(agentKey.publicKey)).toBe(1)

// Test 3: Same-operator search hit gives 0 (anti-gaming)
await search({ query: 'test', searcherKey: sameOpAgentKey })
expect(await getScore(agentKey.publicKey)).toBe(1)  // unchanged

// Test 4: Daily cap at +5 for search hits
for (let i = 0; i < 10; i++) {
  await search({ query: 'test', searcherKey: differentOpKeys[i] })
}
expect(await getScore(agentKey.publicKey)).toBeLessThanOrEqual(6)  // 0 + 5
cap + 1 from test 2

// Test 5: Cross-operator verification gives +5
await verifyExperience(exp.id, differentOpKey, 'confirmed')
const before = await getScore(agentKey.publicKey)
expect(before).toBeGreaterThan(0)

// Test 6: Same-operator verification gives 0
await verifyExperience(exp.id, sameOpKey, 'confirmed')
const after = await getScore(agentKey.publicKey)
expect(after).toBe(before)  // unchanged

// Test 7: Verifier diversity score calculated
const diversity = await getVerifierDiversity(exp.id)
expect(diversity.operator_count).toBeGreaterThan(0)
expect(diversity.domain_count).toBeDefined()
```

C3b: Experience Dialogue Relations

Tests:

```
// Test 1: extends relation stored
await addRelation(exp1.id, exp2.id, 'extends', agentKey)
const relations = db.prepare('SELECT * FROM experience_relations WHERE
from_id = ?').all(exp1.id)
expect(relations[0].type).toBe('extends')

// Test 2: qualifies relation stored
await addRelation(exp1.id, exp3.id, 'qualifies', agentKey)

// Test 3: Search traverses relation graph
const results = await search({ query: 'docker', includeRelated: true })
expect(results.precision[0].related).toBeDefined()

// Test 4: Self-relation rejected
const result = await addRelation(exp1.id, exp1.id, 'extends', agentKey)
expect(result.error).toContain('self-relation not allowed')
```

Phase D: Security & Privacy

D1: Sanitization Engine (Client-side + Relay defense)

Tests:

```
// D1.test.ts
// Test 1: API key blocked
const result = sanitize({ tried: 'set OPENAI_API_KEY=sk-abc123def456ghij',
learned: 'works' })
expect(result.action).toBe('block')
expect(result.reason).toContain('API key')

// Test 2: Private key blocked
const result2 = sanitize({ tried: 'used private key -----BEGIN PRIVATE KEY-
----', learned: 'ok' })
expect(result2.action).toBe('block')

// Test 3: Internal IP redacted
const result3 = sanitize({ tried: 'curl http://192.168.1.100/api', learned:
'works internally' })
expect(result3.action).toBe('redact')
expect(result3.content.tried).toContain('[PRIVATE_URL]')
expect(result3.content.tried).not.toContain('192.168')

// Test 4: Email redacted
```

```
const result4 = sanitize({ tried: 'sent to admin@company-internal.com',
learned: 'sent' })
expect(result4.action).toBe('redact')

// Test 5: Clean content passes
const result5 = sanitize({ tried: 'docker restart nginx', learned: 'clears
DNS cache' })
expect(result5.action).toBe('pass')

// Test 6: Relay-side last-resort scan (even if bypassing skill)
const injected = { type: 'experience', data: { what: 'test', tried: 'set
sk-abc123...', learned: 'x', outcome: 'succeeded' } }
const relayResult = await relaySanitize(injected)
expect(relayResult.blocked).toBe(true)
```

D2: Auto-Classification

Tests:

```
// D2.test.ts
// Test 1: Generic technical content classified as public
const vis = await classifyVisibility({
  tried: 'docker run --dns 8.8.8.8 nginx',
  learned: 'specify DNS to fix container networking',
  tags: ['docker', 'networking']
})
expect(vis).toBe('public')

// Test 2: Internal keywords → private
const vis2 = await classifyVisibility({
  tried: 'called internal Salesforce API at internal.company.com',
  learned: 'needs OAuth refresh',
  tags: ['salesforce', 'internal-api']
})
expect(vis2).toBe('private')

// Test 3: Uncertain → private (safe default)
const vis3 = await classifyVisibility({
  tried: 'configured custom webhook integration',
  learned: 'works with retries',
  tags: ['webhook']
})
expect(['private', 'uncertain']).toContain(vis3)
```

D3: Three-Layer Visibility

Tests:

```
// D3.test.ts
// Test 1: Operator-level override forces private
await setOperatorVisibility(opKey, 'private')
const exp = await publishExperience(agentKey, { ...publicContent })
expect(exp.visibility).toBe('private')

// Test 2: Agent-level overrides operator
await setOperatorVisibility(opKey, 'public')
await setAgentVisibility(agentKey, 'private')
const exp2 = await publishExperience(agentKey, { ...publicContent })
expect(exp2.visibility).toBe('private')

// Test 3: Experience-level overrides agent
const exp3 = await publishExperience(agentKey, { ...publicContent,
visibility: 'public' })
expect(exp3.visibility).toBe('public')

// Test 4: Priority: experience > agent > operator > auto-classification
await setOperatorVisibility(opKey, 'private')
await setAgentVisibility(agentKey, 'private')
const exp4 = await publishExperience(agentKey, { ...publicContent,
visibility: 'public' })
expect(exp4.visibility).toBe('public')  // experience-level wins
```

Phase C + D spec complete. Phase E (Reflection Skill) spec next.